Week 6

Modeling Languages I: MiniZinc and Solver Independence

◎ Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** write a first working MiniZinc model from scratch: declare parameters, decision variables and their domains, state constraints and a `solve` item (`satisfy`, `minimize` or `maximize`), and read a solved instance's output;
- **(H2)** explain what FlatZinc is and what flattening (model transformation) does to a MiniZinc model; read a flattened `.fzn` file well enough to point out where a high-level constraint (e.g. `all_different`) ended up and why parameters disappear from it;
- **(H3)** run one unchanged `.mzn` model against a CP solver (Gecode) and against a MIP solver backend (COIN-BC or HiGHS), compare the results (objective value, solving behaviour), and state precisely what solver independence does and does not guarantee;
- **(H4)** drive MiniZinc from Python using the `minizinc` package (`Model`, `Instance`, `Solver.lookup`), retrieve solutions programmatically, and script a comparison run across more than one solver without leaving Python;
- **(H5)** given a new small combinatorial problem, build a MiniZinc model, verify it compiles and solves correctly on Gecode, run it on at least one MIP backend unchanged, and justify – in the specific terms of the problem's structure – which solver you would choose for it.

Week 5 closed this course's opening arc – five weeks in which every model was built as OR-Tools CP-SAT Python calls – with tree-structured CSPs, tractability, and symmetry-breaking constraints layered on top of an already-correct model. This week makes the turn the course has been heading toward since Week 1: from "one tool, CP-SAT, hard-coded in Python" to "a model written once, in a solver-neutral language, that several different solvers can consume unchanged." Weeks 1 through 5 never separated the *model* you were building from the *program* that ran it, because CP-SAT's Python calls fused the two together by construction. This chapter introduces MiniZinc, a declarative constraint-modeling language, and shows you

that the model – variables, domains, constraints, objective – and the solver – the search/optimization engine that finds values for that model – are two separable things, a distinction the course keeps leaning on through Week 7 (the mathematical-programming view) and beyond.

Four units carry this chapter, and each reuses a problem you either already know or are about to see for the first time. Unit 1 teaches MiniZinc's declarative style and its four building blocks – parameters, decision variables, constraints, and the `solve` item – through the 0/1 knapsack problem, appearing here for the first time in this course. Unit 2 opens the compilation step every MiniZinc model passes through before any solver sees it – flattening into FlatZinc – using the Week 1–4 N-queens model, rewritten in MiniZinc syntax, as its worked example. Unit 3 runs that same kind of unchanged `.mzn` file against Gecode and against a MIP backend on the assignment problem, also appearing for the first time, and states precisely what "solver independence" guarantees and what it does not. Unit 4 closes the chapter by driving MiniZinc from Python with the `minizinc` package and building a short, structural decision guide for choosing a solver family before you ever measure anything.

This chapter deliberately stays out of several topics later weeks or later units own outright. It says nothing about how `mzn2fzn` is implemented internally, or about MiniZinc's own optimizer passes – Unit 2 covers only enough of the pipeline for you to read a flattened model and understand why it looks the way it does. It introduces no `gurobipy` code at all: Gurobi appears exactly once, in Unit 3, as a MiniZinc backend that is registered but fails with a real license error in this environment – the direct Python `gurobipy` API, unaffected by that MiniZinc-specific limitation, is Week 7's subject. It says nothing about duality, shadow prices, sensitivity analysis (Week 10), soft constraints or multi-objective optimization (Week 8), scheduling or interval variables (Weeks 11–12), or infeasibility diagnosis (Week 15) – every model built this chapter is either a pure satisfaction problem or carries a single objective, and every model solves successfully on at least three of the four registered solver targets. Chuffed and an OR-Tools FlatZinc interface are not installed at all in this book's working environment and are never assumed to exist anywhere in this chapter's code, figures or prose.

6.1 Declarative Modeling in MiniZinc

1 • Pre-instruction — Why It Matters

Every model you have built so far – N-queens, Sudoku, map coloring, the social golfer problem – was also, secretly, a Python program: to switch solvers, or to hand your model to a colleague who does not use OR-Tools, you would have had to rewrite it entirely. This unit teaches you to write the same kind of model in a language whose only job is describing the model, not running it, so that, starting in Unit 3, the choice of which program actually solves it becomes a one-line decision instead of a rewrite.

1 • Pre-instruction — Prior Knowledge

Week 5's symmetry-breaking constraints added a lexicographic-ordering boolean expression on top of an already-correct CP-SAT model for the social golfer and BIBD problems – a constraint that was just an extra boolean expres-

sion appended to a model. That is this unit's closest prior example: MiniZinc's `constraint` item is the same idea, written in a different syntax, with no host-language `model.Add(...)` call wrapping it.

Weeks 1 through 5 built every model directly as calls inside a Python program: `model.NewIntVar`, `model.Add(...)` and `solver.Solve(model)` are statements in an imperative host language, executed one after another, even though the constraint model those statements build is itself declarative in spirit. This unit introduces a language that drops the host program entirely and lets a model stand on its own.

6.1.1 Declarative modeling: stating what, not how

MiniZinc: A high-level, solver-independent declarative language for expressing constraint satisfaction and optimization models, compiled to FlatZinc before being solved.

A **MiniZinc** model is *declarative*: it states *what* a solution must satisfy – and, optionally, what makes one solution better than another – without saying anything about *how* to search for it. Nowhere in a `.mzn` file do you write a search loop, a backtracking step or a branching rule; you write down variables, their domains and the constraints they must jointly satisfy, and a separate program (the solver) figures out how to search. Contrast this with Week 1's `model.NewIntVar(0, n-1, "q0")` followed by `model.Add(q0 != q1)`: the model those calls build is also declarative in the sense that it states constraints rather than a search procedure, but every one of those calls still executes as an instruction in a Python program you wrote and control. MiniZinc removes that host program: a `.mzn` file has no notion of "execute this line, then that line" at all – it is read as a whole, and only ever describes a model, never a sequence of actions.

Analogy

A recipe card lists ingredients and the finished dish's requirements – "must serve four, must be vegetarian" – without saying which brand of oven cooks it or in what order a particular kitchen happens to prepare side dishes. A MiniZinc model lists variables, domains and constraints without saying which program searches for a solution: the same recipe card works in any kitchen with an oven, and the same `.mzn` file works with any installed solver.

A common misconception at this point is assuming that "declarative" means "simpler" or "less expressive" than a Python CP-SAT script. It is neither – MiniZinc's `constraint` items can express arbitrarily complex boolean expressions over variables, exactly like `model.Add(...)` can. What changes is not expressive power but where the search machinery lives: outside the model file entirely, in whichever solver you eventually choose, rather than baked into the same file as the model.

6.1.2 The four building blocks of a `.mzn` file

Every MiniZinc model, however large, is built from exactly four kinds of items.

parameter (MiniZinc): A fixed, known quantity declared in a MiniZinc model (`int`, `float`, `bool`, `set of int`, or an array of these), typically supplied by a `.dzn` data file.

The first of the four is a **parameter (MiniZinc)**: a plain `int`, `float`, `bool` or `set of int`, or an array built from any of those. A parameter's value is data the model receives, never a value the solver chooses – it is the MiniZinc-side equivalent of a plain Python number sitting next to your Week 1–5 CP-SAT code, rather than anything passed to `NewIntVar`.

The second building block is what a parameter is not: an unknown. A **decision variable (MiniZinc)** is written with the keyword `var` in front of its type (`var int`,

`var bool`, `var set of int`) and is exactly the counterpart of a CP-SAT `NewIntVar` or `NewBoolVar`: an unknown the solver assigns a value to, from within a stated domain, rather than a value the model already knows before solving starts. Distinguishing the two – which of a problem's quantities are given data and which are unknowns to be found – is the very first question a MiniZinc model asks, the same first question Week 1 Unit 1 taught you to ask of a CSP's variables and domains.

Third, a `constraint` item is an arbitrary boolean expression over parameters and decision variables that every solution must satisfy – Week 5's onbilgi note above is the direct bridge into this piece.

Definition: Constraint item (MiniZinc)

A MiniZinc statement of the form `constraint <boolean expr>;` declaring a boolean expression over parameters and decision variables that every solution must satisfy. A model may contain any number of constraint items; a solution must satisfy all of them simultaneously.

solve item: The single MiniZinc statement declaring whether a model is a satisfaction problem (`solve satisfy;`) or an optimization problem (`solve minimize expr;` / `solve maximize expr;`).

Fourth, and exactly one per model, a **solve item** tells the solver what kind of problem this is: `solve satisfy;` for a pure constraint satisfaction problem (any solution will do), `solve maximize expr;` or `solve minimize expr;` for an optimization problem, where `expr` is the objective to optimize. Nothing else in a `.mzn` file plays this role – a model with no `solve` item, or with more than one, does not compile.

An ordinary mistake at this stage, especially coming straight from five weeks of CP-SAT, is marking every declared quantity `var` out of habit – writing `var int: capacity;` for a knapsack's capacity, say, when its numeric value is supplied as data and never chosen by the solver. If a quantity's value is known before you ever run the model, declare it as a plain parameter, not a decision variable; reserve `var` strictly for the unknowns the solver is meant to determine.

6.1.3 Separating a model from its data: `.mzn` and `.dzn`

.dzn file: A MiniZinc data file supplying concrete values for a model's parameters, kept separate from the `.mzn` model file so one model can be reused across many data instances.

A `.mzn` file can declare a parameter without giving it a value – `int: n;` on its own, with no `= ...` – and a companion **.dzn file** supplies the actual number, `n = 5;`. This is the same model/instance separation a Sudoku *puzzle* (the 81 given and blank cells) has from the Sudoku *rules* (one variable per cell, `all_different` on every row, column and box) – except MiniZinc makes the split explicit as two files rather than leaving it implicit inside one Python script, the way Weeks 1 through 5 hard-coded a specific instance's numbers directly next to the model that used them.

Analogy

A travel-itinerary template – "visit these three cities in an order that avoids weekend closures" – stays the same every time you travel; only the dates, city list and closure days change from trip to trip. One template, many trips, exactly like one `.mzn` model file reused across many `.dzn` data files.

6.1.4 Worked running example: the 0/1 knapsack problem

knapsack problem: The classic optimization problem of choosing a subset of items, each with a weight and a value, to maximize total value without exceeding a capacity on total weight.

This unit's running example is the classic 0/1 **knapsack problem**: given n items, each with a `weight` and a `value`, and a knapsack `capacity`, choose a subset of items – one `var bool` per item – whose total weight does not exceed the capacity and

whose total value is maximized. The parameters are declared exactly as Unit 1's building-block list above prescribes:

```
int: n;
int: capacity;
array[1..n] of int: weight;
array[1..n] of int: value;
```

Four plain (non-`var`) declarations – n, `capacity`, and the two arrays – match exactly what a `.dzn` file for this model must supply. The Unit 1 practice-task instance used throughout this unit's examples fits in four short lines:

```
n = 4;
capacity = 7;
weight = [2, 3, 4, 5];
value = [3, 4, 5, 6];
```

The decision variables – one boolean per item, `true` if that item is selected – come next:

```
array[1..n] of var bool: x;
```

The capacity constraint sums each selected item's weight and bounds that sum by `capacity`:

```
constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;
```

Finally, the `solve` item states this is a maximization problem, together with an `output` item that formats the solved instance's answer as text instead of leaving MiniZinc to print raw variable values:

```
solve maximize sum(i in 1..n)(value[i] * x[i]);

output ["selected items: \(x)\n",
      "total value: \(sum(i in 1..n)(value[i] * x[i]))\n"];
```

The `\(...\)` syntax inside the string literals is string interpolation: it substitutes a decision variable's solved value directly into the printed text, so the model controls exactly what a solved instance shows on screen, rather than a solver dumping every variable's raw value in whatever order it happens to store them.

Case Study 6.1: Solving the knapsack model on Gecode

Running `minizinc --solver Gecode knapsack.mzn knapsack.dzn` against the four-item instance above produces this real, captured output:

Expected Output

```
selected items: [false, true, true, false]
total value: 9
```

Reading the `x` array against the `weight`/`value` arrays above: items 2 and 3 are selected (weights 3 and 4, total 7, exactly at capacity), items 1 and 4 are left out, and the total value is $4 + 5 = 9$ – selecting item 1 instead of item 2 or 3 would waste capacity on a lower value-per-weight item, so 9 is indeed the best achievable total under a capacity of 7. Note the harmless compiler warning this particular model produces (visible on the terminal, not in the file above): MiniZinc's own standard library ships a global constraint file also named `knapsack.mzn`, so this model's file name overrides a standard-library file of the same name. The warning is expected and does not affect the solved output; it is not a bug

to chase down.

6.1.5 Now you try

3 • Participation **Now You Try**

Given $n = 4$ items with weights `[2, 3, 4, 5]` and values `[3, 4, 5, 6]`, and knapsack capacity `7`, write the MiniZinc decision-variable declaration and the capacity constraint (do not write the full model).

Feedback. The decision-variable declaration is one boolean per item:

```
array[1..n] of var bool: x;
```

and the capacity constraint is:

```
constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;
```

The multiplication `weight[i] * x[i]` works because MiniZinc treats `bool` as `0/1` in arithmetic contexts – when `x[i]` is `false` it contributes `0` to the sum, and when it is `true` it contributes the full `weight[i]` – so no separate "if selected" branch is needed anywhere in the constraint.

Check Yourself 6.1

In MiniZinc, what is the difference between a *parameter* and a *decision variable*, and which of the two would `capacity` be in the knapsack model above?

5 • Follow-through **Transfer — Where Else?**

The parameter/variable/constraint/solve structure you just used for knapsack is the same structure Unit 3 reuses, unchanged, for the assignment problem, and the same structure the lab sheet asks you to reuse for a problem you have not seen modeled before. Once you can name which of your problem's quantities are parameters and which are decision variables, the rest of a MiniZinc model follows close to mechanically – and that naming step is exactly the same first question Week 1 Unit 1 taught you to ask of a CSP's variables and domains, just answered in a new syntax.

6.2 FlatZinc and Model Transformation

1 • Pre-instruction **Why It Matters**

You already know, from Week 1, exactly what N-queens' variables, domains and constraints are on paper. This unit answers a question you have never had to ask about a CP-SAT model, because CP-SAT hides the answer inside its own library: what does your model actually look like once a compiler has finished translating it into the form a solver reads?

1 • Pre-instruction **Prior Knowledge**

Week 1 Unit 3 formulated N-queens as one variable per row, domain $\{1, \ldots, n\}$, and three `AllDifferent` calls – columns and both diagonals. Week 1 Unit 2 also introduced the idea of an *auxiliary variable*, a variable added purely to make some constraint easier to state without changing what counts as a solution. This unit reuses the exact same N-queens model, now written in MiniZinc, and reuses "auxiliary variable" to explain variables a compiler introduces that you never wrote yourself.

Unit 1 handed you a working `.mzn` file and a real solved output, but skipped over a step that happened silently in between: no solver anywhere reads a `.mzn` file directly. This unit opens that step up.

6.2.1 Flattening: from `.mzn` to FlatZinc

FlatZinc (`.fzn`): The low-level, solver-agnostic intermediate format that every MiniZinc-compatible solver actually reads; produced by flattening a `.mzn` model.

A **FlatZinc (`.fzn`)** file is a low-level, solver-agnostic intermediate format built from a small, fixed set of predicates – `int_lin_le`, `fzn_all_different_int`, `array_var_int_element`, and a few dozen more. Every solver MiniZinc supports – Gecode, COIN-BC, HiGHS, and, in principle, Gurobi – reads this same intermediate format, never the original `.mzn` source; that single shared format is precisely what makes Unit 3's solver-independence claim possible at all, since two different solver programs can only agree to read one another's input if that input is standardized.

flattening (model transformation): The compilation step (`mzn2fzn`) that translates a high-level MiniZinc model, together with its data, into FlatZinc: substituting parameter values, translating global constraints into solver-understood predicates, and introducing any auxiliary variables the translation needs.

The compilation step that produces a `.fzn` file from a `.mzn` model is called **flattening (model transformation)**. MiniZinc's own `mzn2fzn` tool takes a `.mzn` model plus its `.dzn` data and produces a `.fzn` file with three concrete changes. First, every parameter's value is substituted directly into the file – `n` disappears as a named quantity, replaced everywhere by the actual number from the `.dzn` file. Second, every high-level global constraint, such as `all_different(q)`, is mapped to whichever FlatZinc predicate the target solver's own library defines for it – or, for a solver without a native version of that constraint, decomposed into a conjunction of simpler predicates. Third, the flattener may introduce auxiliary variables of its own that were never written by you, the modeler, needed purely as internal bookkeeping for the translation – the same idea Week 1 Unit 2 taught you by hand, now performed automatically by a compiler instead.

Analogy

A travel agent converts a plain-language itinerary – "fly to city A on Tuesday, then city B on Thursday" – into a specific airline's own booking codes. The itinerary you wrote in your own words never changes, but each airline's internal booking system needs its own coded format before it can act on it. A MiniZinc model is translated into whichever FlatZinc predicates the chosen solver's own library understands, without you rewriting the itinerary by hand.

Figure 6.1 lays out the pipeline end to end: a `model.mzn` file plus a `data.dzn` file are compiled (`minizinc --compile`, that is, `mzn2fzn` running internally) into a `model.fzn` file; a chosen solver's own FlatZinc interpreter reads that file and produces a stream of solutions; and MiniZinc's own solution printer applies the model's `output` item – the same `output` item Unit 1 wrote for the knapsack model – to turn raw solved values back into the text you actually see on screen. Nothing about the

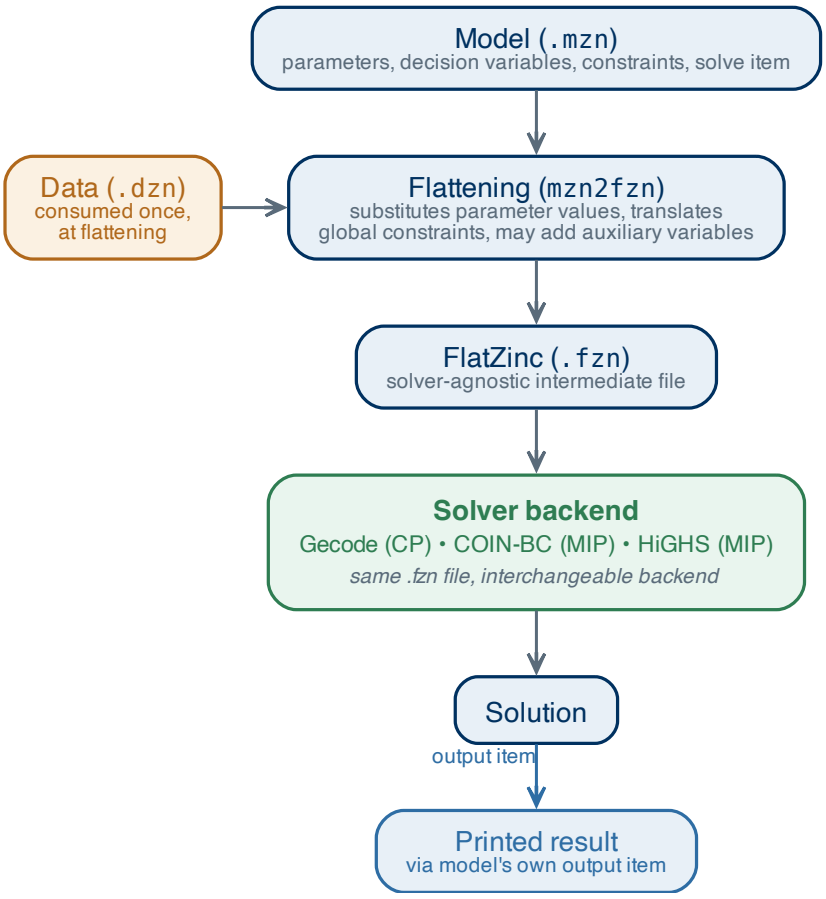


Figure 6.1: The MiniZinc compilation pipeline: a model and data file are flattened into solver-agnostic FlatZinc, solved by an interchangeable backend, and printed back through the model's own `output` item.

`output` item survives inside the `.fzn` file itself; it is applied afterward, by MiniZinc's own tooling, once the solver has returned an answer.

6.2.2 A version-specific library detail: `include`

A concrete, small detail of this exact environment (MiniZinc 2.10.1) makes the point that a modeling language has its own library and its own versioning, entirely separate from anything solver-specific: `all_different` is no longer auto-included in this version. A model that calls it must start with the line `include "all_different.mzn";` (or the broader `include "globals.mzn";`, which pulls in every global constraint at once) – omit it, and `mzn2fzn` reports an undefined identifier at flattening time, before the model ever reaches a solver.

Common Pitfall

Forgetting the `include` line is an easy mistake to make once you already know, from Week 1, that `AllDifferent` is simply available in CP-SAT with no import step. MiniZinc is not CP-SAT: its global constraints live in a standard library of `.mzn` files that must be explicitly pulled in, and `all_different` is one of them in this version. Both `nqueens.mzn` and `assignment.mzn` carry `include "all_different.mzn";` as their very first non-comment line for exactly this reason.

6.2.3 Worked running example: N-queens, flattened

This unit's running example rewrites the Week 1–4 N-queens model in MiniZinc. The declarations follow Unit 1's building-block pattern directly:

```
int: n;
array[1..n] of var 1..n: q;
```

Three `all_different` calls state, in order, that no two queens share a column, and that no two queens share either diagonal – the `q[i] + i` and `q[i] - i` transformations are exactly the diagonal encoding Week 1 Unit 3 derived on paper, unchanged:

```
constraint all_different(q);
constraint all_different([q[i] + i | i in 1..n]);
constraint all_different([q[i] - i | i in 1..n]);
```

`solve satisfy;` declares this a pure satisfaction problem – any board respecting all three constraints is an acceptable answer, and there is no objective to maximize or minimize:

```
solve satisfy;

output ["queen columns per row: \(q)\n"];
```

Compiling this model for a 6-queens instance (`n = 6` in the accompanying `.dzn`) with `minizinc --compile` produces a real `.fzn` file. A short, real excerpt of it, read side by side with the `.mzn` source above, shows exactly what flattening did:

```
Terminal
$ minizinc --compile -o nqueens.fzn --solver Gecode nqueens.mzn
↪  nqueens.dzn
$ cat nqueens.fzn
```

Expected Output

```
% Generated by MiniZinc 2.10.1
% Solver library: /opt/homebrew/Cellar/gecode/6.4.0/share/minizinc/gecode

predicate fzn_all_different_int(array [int] of var int: xs);
...
array [1..6] of var int: q :: output_array([1..6]) = [...];
array [1..6] of var int: X_INTRODUCED_13_ :: var_is_introduced = [...];
array [1..6] of var int: X_INTRODUCED_20_ :: var_is_introduced = [...];
constraint fzn_all_different_int(q);
constraint fzn_all_different_int(X_INTRODUCED_13_);
constraint fzn_all_different_int(X_INTRODUCED_20_);
...
solve  satisfy;
```

Four things to read off this excerpt against the `.mzn` source. First, `n` is gone entirely – every array in the file is written with the concrete bound `[1..6]`, the number substituted directly from `nqueens.dzn`. Second, the literal identifier `all_different` does not survive: all three calls became `fzn_all_different_int`, the `fzn_` prefix marking a FlatZinc builtin predicate and the `_int` suffix marking the variable type it applies to. Third, the two diagonal expressions `q[i] + i` and `q[i] - i` were not left as expressions inside the `fzn_all_different_int` call; each one became its own array of freshly introduced variables – `X_INTRODUCED_13_` and `X_INTRODUCED_20_` – tied by further (elided here) constraints to `q`. These are exactly the compiler-introduced auxiliary variables Unit 2's onbilgi promised: you never wrote a variable named `X_INTRODUCED_13_`, but the flattener needed one to hold the value of `q[i] + i` for each i before it could hand that array to `fzn_all_different_int`. Fourth, `q` itself survives under its own name only because of the `output_array` annotation the flattener attaches to it – that annotation is what lets MiniZinc's own solution printer find `q`'s solved values again once the solver returns them, so the `output` item's `\(q\)` interpolation can print something recognizable.

Tip

Reading a `.fzn` excerpt does not require understanding every line. Look for three landmarks: array declarations with a concrete integer bound (parameters substituted), predicate names carrying an `fzn_` prefix (global constraints translated), and variable names carrying `X_INTRODUCED_` (auxiliary variables the compiler added). Everything else is detail you can safely skim past.

6.2.4 Now you try

3 • Participation **Now You Try**

Run `minizinc --compile -o nqueens.fzn nqueens.mzn` (with `n = 6` in the accompanying `.dzn`) and open the resulting `.fzn` file. Find the FlatZinc line(s) that correspond to your `all_different(q)` constraint on the columns, and state whether the literal string `all_different` still appears.

Feedback. For Gecode's FlatZinc target, the line is `constraint fzn_all_different_int(q);` – the `fzn_` prefix marks it as a FlatZinc builtin predicate, and the `_int` suffix marks the variable type it operates over; this is

confirmed real against this environment's own captured `.fzn` output above, not a generic guess. The literal MiniZinc identifier `all_different` does not survive flattening unchanged: it is translated to whichever FlatZinc predicate name the target solver's own library defines for that global constraint, which is why a different solver's FlatZinc library could, in principle, choose a different predicate name for the same constraint.

Check Yourself 6.2

Why does a flattened `.fzn` file not contain the literal MiniZinc keyword `all_different`, even when the original `.mzn` model used `constraint all_different(q);`?

5 • Follow-through Transfer — Where Else?

Understanding that "model" and "FlatZinc program" are two different things, connected by a compilation step, is exactly what makes Unit 3's claim – run the same `.mzn` file on two different solvers – unsurprising rather than magical: two solvers just read two different `.fzn` files, both flattened from the one `.mzn` source you wrote once. The next unit puts that fact to work directly, on a problem with an objective this time.

6.3 Running One Model on Gecode and a MIP Backend, Unchanged

1 • Pre-instruction Why It Matters

Two students each own a copy of the exact same `assignment.mzn` file. One types `minizinc --solver Gecode assignment.mzn`; the other types `minizinc --solver COIN-BC assignment.mzn`. Neither of them edits a single line of the model – and yet, under the hood, one program is about to do backtracking search with constraint propagation and the other is about to solve a sequence of linear programs with branch-and-bound. This unit is about what happens when you make that choice a command-line flag instead of a rewrite – and, just as importantly, about what happens when that flag names a solver that is listed but not actually usable, which is a different failure from anything wrong in your model.

1 • Pre-instruction Prior Knowledge

Unit 2's flattening pipeline told you that "the same `.mzn` on two solvers" really means "two different `.fzn` files, both compiled from one `.mzn` source." That fact is the reason this unit's guarantee – a shared optimal value – is a fact about compilation correctness, not a coincidence: both `.fzn` files describe the exact same mathematical problem, just phrased in the vocabulary each target solver understands.

Every claim in this unit rests on one environment fact worth stating plainly before anything else. `minizinc --solvers` on this machine (MiniZinc 2.10.1,

Homebrew) *lists* four solver targets – Gecode and three MIP backend registrations, Gurobi, HiGHS and COIN-BC – but being listed is not the same as being usable. Gecode, HiGHS and COIN-BC all actually solve a model here. Gurobi's MiniZinc plugin fails outright with `No Gurobi license found`: MiniZinc's own Gurobi backend needs a full Gurobi license file, and the restricted license that lets `gurobipy` itself run (first used directly in Week 7) does not cover it. Chuffed and an OR-Tools FlatZinc target are not installed at all in this environment – a separate, unrelated gap. Every "solver independence" run in this book therefore means **Gecode against COIN-BC and/or HiGHS**; Gurobi appears exactly once below, specifically to show that "registered" and "usable" are two different claims, with real captured evidence rather than a hypothetical.

6.3.1 Solver independence, precisely

Solver independence: The property that one MiniZinc model (source unchanged) can be solved by more than one solver backend, all agreeing on feasibility and, for optimization problems, on the optimal objective value.

Solver independence names the property this unit demonstrates: the same `.mzn` (plus its `.dzn`) source file, with no edits, is handed to `minizinc --solver <name>` for two different kinds of target, and both return a solution to the same problem. What is guaranteed to match is precise and narrow: the optimal objective value, when the problem has one, and feasibility or infeasibility. What is *not* guaranteed to match is everything about *how* that answer was reached – solving time, which specific optimal solution is reported when several exist with the same objective value, and internal search behaviour.

CP solver: A solver, such as Gecode, that finds solutions via constraint propagation combined with backtracking search.

The first of the two targets this unit compares is a **CP solver** – Gecode is this book's example throughout. It finds solutions via constraint propagation combined with backtracking search, the same general style of algorithm Week 2 already opened up for you inside CP-SAT, just now living behind a MiniZinc `.fzn` interface instead of a Python call. Nothing about *how* Gecode searches changed by arriving through MiniZinc rather than through `cp_model.CpSolver()` – only the language the model in front of it was written in changed.

MIP backend: A solver, such as COIN-BC or HiGHS (or Gurobi, in general – currently license-blocked in this environment's MiniZinc plugin), that solves an optimization problem via LP relaxation combined with branch-and-bound.

The second target is a **MIP backend** – COIN-BC and HiGHS are this book's two working examples. It solves an optimization problem via LP relaxation combined with branch-and-bound, a genuinely different algorithm family from Gecode's, previewed here in name only and fully explained once Week 7 opens `gurobipy` directly. A CP solver and a MIP backend, then, find solutions by genuinely different routes – propagation plus backtracking search versus LP relaxation plus branch-and-bound – and "solver independence" is a claim about their agreement on the *answer*, never a claim that they work the same way to get there.

Analogy

Handing one grocery list to two different delivery services: the list itself never changes. One service sends a runner who walks the aisles picking items one at a time, much like propagation plus search; the other plans an optimized shopping route across several stores before buying anything, much like LP relaxation plus branch-and-bound. Both come back with the same groceries, by genuinely different methods, and neither required you to rewrite the list.

Analogy

A bike courier and a freight-routing service can both deliver the same package, but each is naturally suited to a different kind of job: a bike courier is fast and

flexible for a single point-to-point job with many local restrictions – a combinatorial, constraint-heavy job, like N-queens – while a freight-routing service is built to optimize a numeric cost across many possible routes at once – a linear-objective job, like the assignment problem below.

A frequent misconception at this point is expecting an *identical* run in every respect once the optimal value matches – assuming, for instance, that solve time should also match, or that two solvers reporting the same cost must have explored the same search states internally. Neither follows from solver independence as defined above; only the optimal value (or feasibility) is the guarantee, and the rest of this unit's worked example makes that distinction concrete with real numbers.

6.3.2 Worked running example: the assignment problem

assignment problem: The classic optimization problem of pairing each of n workers with exactly one of n tasks, one-to-one, minimizing total assignment cost.

This unit's example is the **assignment problem** – deliberately chosen over N-queens because it carries an objective, so "the optimal *value* matches across solvers" is a concrete, checkable claim rather than a bare satisfiability check. The parameters are n and an $n \times n$ cost matrix:

```
int: n;
array[1..n, 1..n] of int: cost;
```

One decision variable per worker, holding the task index it is assigned:

```
array[1..n] of var 1..n: assign;
```

A single `all_different` constraint is the entire feasibility requirement – no two workers may share a task, which, combined with one variable per worker, also guarantees every task gets exactly one worker:

```
constraint all_different(assign);
```

The objective sums each worker's cost for the task it is assigned, and `solve minimize` closes the model:

```
var int: total_cost = sum(i in 1..n)(cost[i, assign[i]]);
solve minimize total_cost;

output ["assignment: \(assign)\n",
      "total cost: \(total_cost)\n"];
```

For the 4×4 cost matrix in `assignment.dzn`, running this unchanged model against all three working solvers produces the real, captured outputs below.

Terminal

```
$ minizinc --solver Gecode --statistics assignment.mzn assignment.dzn
```

Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
...
%%%mzn-stat: solveTime=5e-05
%%%mzn-stat: nodes=13
%%%mzn-stat: failures=2
...
```

Terminal

```
$ minizinc --solver COIN-BC --statistics assignment.mzn assignment.dzn
```

Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
...
%%%mzn-stat: objective=13
%%%mzn-stat: nodes=0
%%%mzn-stat: solveTime=0.0012
...
```

Terminal

```
$ minizinc --solver HiGHS --statistics assignment.mzn assignment.dzn
```

Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
...
%%%mzn-stat: objective=13
%%%mzn-stat: nodes=1
%%%mzn-stat: solveTime=0.0044
...
```

All three report the identical assignment `[2, 1, 3, 4]` and the identical total cost 13 – worker 1 to task 2, worker 2 to task 1, worker 3 to task 3, worker 4 to task 4. That the assignment itself matches here is not part of the solver-independence guarantee (it is only guaranteed for the *cost*); it happens because this particular instance has a single, unique optimal assignment, so there is no tie for a solver to break differently. What does differ, exactly as the guarantee above predicted, is each solver's own internal statistics – Gecode reports `nodes=13` (search-tree nodes) and `failures=2` in its own propagation-plus-search vocabulary; COIN-BC and HiGHS instead report `nodes` in a branch-and-bound sense (0 and 1 respectively for this small instance) and never mention "failures" at all, because the concept does not exist in their algorithm.

Caution

Running the same command with `--solver Gurobi` on this environment produces a real, captured failure, not a fourth working comparison point:

Terminal

```
$ minizinc --solver Gurobi --statistics assignment.mzn assignment.dzn
```

Expected Output

```
Error: error:   MIPGurobiWrapper runtime error:  No Gurobi license
found (user tolgaberber, host Tolga-M3-MacBook-Pro.local, hostid
5f3bca5c, cores 16)
Message from caller: Could not open GUROBI environment.
exit code: 1
```

This is a license problem specific to MiniZinc's own Gurobi plugin, not a problem with the model above (the same model solves correctly on three other backends) and not the restricted-license problem `gurobipy` itself has – `gurobipy`'s bundled restricted license lets it run directly, as Week 7 shows, but MiniZinc's separate Gurobi backend needs a full license file that is not present here. Treat a `minizinc --solvers` listing as an inventory of what is *registered*, never as proof that a given name will actually solve anything.

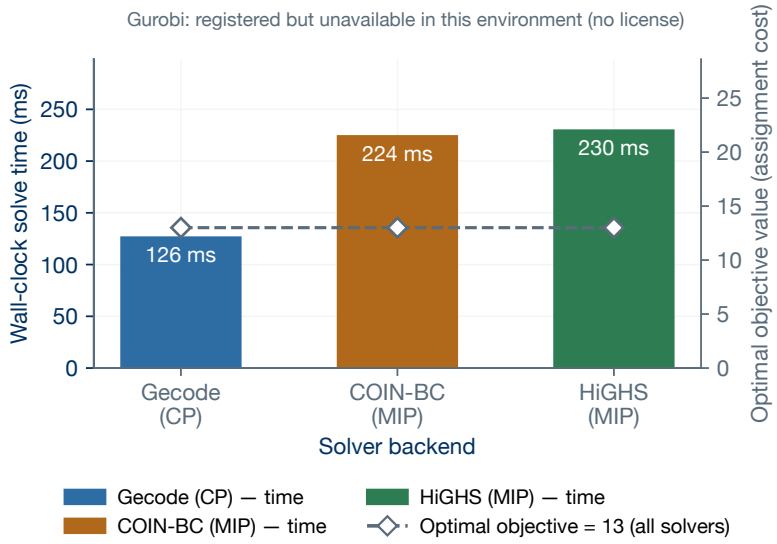


Figure 6.2: Wall-clock solve time by solver backend, with the shared optimal objective value.

Figure 6.2 plots the wall-clock time each working solver took on this instance, measured end to end (process start to printed answer, not just the solver's own internally reported `solveTime`): Gecode around 126 ms, COIN-BC around 224 ms, and HiGHS around 230 ms, all three bars landing on the same optimal cost of 13. Gecode's wall-clock advantage here is specific to this small instance and this measurement, not a general law that CP solvers are always faster than MIP backends – Unit 4's decision guide returns to that question structurally rather than by timing alone.

6.3.3 Now you try

3 • Participation **Now You Try**

For a 4×4 assignment instance, running on Gecode reports optimal cost `13`. You then run the identical `.mzn`/`.dzn` pair on HiGHS. What value do you expect for the optimal cost, and what – if anything – might differ between the two runs?

Feedback. The optimal cost must also be `13` – both solvers are finding the true optimum of the same mathematical problem, and an assignment problem has one well-defined optimal value regardless of which correct algorithm finds it. What may differ is which optimal assignment is reported, if more than one achieves cost `13` (not the case for this particular instance, which has a unique optimum), and how long each solver took, since Gecode's search order and HiGHS's simplex-plus-branch-and-bound path are unrelated procedures that happen to agree on the answer.

Check Yourself 6.3

You run the same `assignment.mzn/assignment.dzn` pair on Gecode and on HiGHS. What is guaranteed to be identical between the two runs, and what is not?

5 • Follow-through **Transfer — Where Else?**

This exact pattern – same model, different backend, same optimal value – is what lets Week 7 introduce `gurobipy` and later weeks move freely between CP-SAT-style and MIP-style views of a problem without re-deriving a model from scratch each time. Solver independence in MiniZinc this week is this course's first, smallest example of a habit you will keep using for the rest of the semester: separating "is the model correct" from "which engine should solve it," including in your own future research code outside this course.

6.4 MiniZinc from Python; Choosing a Solver

1 • Pre-instruction **Why It Matters**

Typing `minizinc --solver Gecode model.mzn`, reading the output, then retyping the command with `--solver COIN-BC` and again with `--solver HiGHS`, copying numbers into a spreadsheet by hand – this does not scale past two or three comparisons, and it gives you no automatic way to notice, before you waste time reading an error message, that a fourth solver name is registered but will not actually run. This unit gets Python to do the retyping, the copying, and the "does this solver actually work" check for you, the same way your `tests/` scripts have been checking your CP-SAT models automatically since Week 1.

1 • Pre-instruction **Prior Knowledge**

Unit 3's solver-independence claim – same `.mzn`, different `--solver` flag, matching optimal value – is exactly what this unit expresses again, only as Python objects instead of a shell command. Nothing about *what* is being compared is new here, only *how* you drive it.

Every command-line run in Units 1 through 3 has a direct Python counterpart: the `minizinc` package (the same one Week 1's setup already pinned, `minizinc==0.10.0`) exposes MiniZinc's model, solver-lookup and solve steps as ordinary Python objects, letting you script exactly the kind of multi-solver comparison Unit 3 did once, by hand, on the command line.

6.4.1 The `minizinc` package's three core objects

Model (Python `minizinc` package): The Python object representing a loaded MiniZinc model, ready to be bound to a solver as an `Instance`.

A **Model (Python `minizinc` package)** object loads a `.mzn` file (and, via `add_file`, a companion `.dzn`), exactly the source you already wrote by hand in Units 1 through 3:

```
knapsack_model = minizinc.Model(MZN_DIR / "knapsack.mzn")
knapsack_model.add_file(MZN_DIR / "knapsack.dzn")
nqueens_model = minizinc.Model(MZN_DIR / "nqueens.mzn")
nqueens_model.add_file(MZN_DIR / "nqueens.dzn")
```

Loading a model this way creates no solved answer yet – it only reads and parses the `.mzn`/`.dzn` pair, the Python-side counterpart of opening the file in a text editor. To actually solve something, a `Model` must first be paired with a solver, which is where the next two objects come in.

Instance (Python `minizinc` package): A MiniZinc `Model` bound to a specific `Solver`, ready to be solved from Python.

An **Instance (Python `minizinc` package)** binds a loaded `Model` to one specific solver. That solver is not named as a bare string handed straight to `Instance` – it is looked up first, by name, through a dedicated function this package provides for exactly that purpose.

The lookup step uses **Solver.lookup**, called with the exact same solver names Unit 3 already typed after `--solver` on the command line, now passed as a Python string instead:

Solver.lookup: The `minizinc` Python package function that finds an installed solver by the name reported by `minizinc --solvers`.

```
solver = minizinc.Solver.lookup("gecode")
instance = minizinc.Instance(solver, model)
start = time.perf_counter()
result = instance.solve()
elapsed_ms = (time.perf_counter() - start) * 1000
rows.append(("Gecode", str(result.status), elapsed_ms, summarize(result)))
```

Calling `instance.solve()` is the Python-side counterpart of the whole `minizinc --solver <name> model.mzn` command line: it runs `mzn2fzn` internally, hands the resulting FlatZinc to the chosen solver, and returns a `Result` object whose `.solution` attribute exposes decision-variable values the same way `solver.Value(var)` did for CP-SAT throughout Weeks 1 through 5, and whose `.status` reports whether that solution is optimal, merely feasible, or absent.

Analogy

A universal remote programs one set of channel presets once, then points the same remote at a different brand of television by selecting its device code. The presets – your model – do not change; only which device – solver – the remote

currently controls does. `Model` plays the role of the presets, and each call to `Solver.lookup` plays the role of selecting a new device code.

6.4.2 Probing before relying on a backend

Unit 3 already showed that a name listed by `minizinc --solvers` is not guaranteed to actually solve anything – Gurobi, in this environment, is exactly such a case. A script that blindly loops over solver names and calls `.solve()` on each would crash the moment it reached Gurobi, exactly where a careless script would be least convenient to have crash: partway through a batch comparison, with earlier results potentially lost. `minizinc_python.py` instead wraps each MIP-backend attempt in a `try/except` around `minizinc.MiniZincError`, recording "unavailable" plus the real error reason instead of letting the whole run crash:

```
for backend in MIP_BACKENDS:
    try:
        solver = minizinc.Solver.lookup(backend)
        instance = minizinc.Instance(solver, model)
        start = time.perf_counter()
        result = instance.solve()
        elapsed_ms = (time.perf_counter() - start) * 1000
        rows.append((backend, str(result.status), elapsed_ms, summarize(result)
            ))
    except minizinc.MiniZincError as error:
        rows.append((backend, "unavailable", 0.0, str(error).splitlines()[0]))
```

Common Pitfall

Skipping this `try/except` guard and simply calling `Solver.lookup("gurobi")` followed by `instance.solve()` looks harmless when every backend you have tried so far happens to work – until the first backend that is registered but not actually usable (Gurobi here) raises `minizinc.MiniZincError` and takes the whole script down with it, discarding every comparison result already computed. Probing with a guard around each backend, not just around the whole script, is what lets the comparison table below print COIN-BC's and HiGHS's real results *and* Gurobi's failure reason in the same run.

Finally, the comparison table itself is ordinary Python string formatting over the collected rows – nothing solver-specific happens here at all:

```
print(f"\n{problem_name}:")
print(f"{'solver':<10} {'status':<20} {'time_ms':>10} summary")
for solver_name, status, ms, summary in rows:
    print(f"{solver_name:<10} {status:<20} {ms:>10.2f} {summary}")
```

Case Study 6.2: Running the Unit 4 comparison script

Running `minizinc_python.py` against this week's knapsack instance produces this real, captured output:

Expected Output

```
knapsack (n=4, capacity=7):
solver     status                  time_ms  summary
Gecode     OPTIMAL_SOLUTION         117.83  objective=9
coin-bc    OPTIMAL_SOLUTION         220.22  objective=9
highs      OPTIMAL_SOLUTION         221.70  objective=9
gurobi     unavailable                0.00    MIPGurobiWrapper
↪  runtime
error:  No Gurobi license found (user tolgaberber, host
Tolga-M3-MacBook-Pro.local, hostid 5f3bca5c, cores 16)
```

All three working solvers agree on `objective=9` – the same value Unit 1's Gecode run reported by hand – while Gurobi's row reports its real captured license error instead of a fabricated result, exactly the "listed versus usable" distinction Unit 3 first demonstrated on the command line, now handled automatically inside a script rather than crashing it.

6.4.3 Choosing which solver to run

Units 1 through 3 already showed, empirically, that a CP solver and a MIP backend can both solve the same model correctly, at different speeds and by different routes. That observation supports a short, structural decision guide – a starting heuristic to test empirically, as Unit 3 already did once, not a theorem (Week 7 develops the full "why"). Problems whose feasibility structure dominates – many hard combinatorial constraints, an "any solution" or small-domain flavour, such as N-queens' three `all_different` calls with no objective at all – tend to suit a CP solver like Gecode's propagation-plus-search well. Problems whose objective and largely linear/arithmetic constraints dominate – such as the assignment problem's linear cost sum, and later weeks' production planning – tend to suit a MIP backend's LP-relaxation-plus-branch-and-bound well.

Figure 6.3 lays this guide out as a structural question, not a lookup table to memorize: ask whether a new problem's difficulty comes mainly from satisfying many combinatorial constraints or mainly from optimizing a linear cost, and let that answer point you toward a CP solver or a MIP backend as a starting choice – one you can, and should, still verify empirically the way Unit 3 did.

Complexity (big-O)

This decision guide is a structural heuristic about where a problem's difficulty tends to concentrate, not a runtime guarantee – nothing here claims a proven worst-case bound favouring one solver family over the other for either problem shape. Unit 3's measured wall-clock numbers (Figure 6.2) are real for this one small instance and this one machine; they illustrate the guide, they do not prove it.

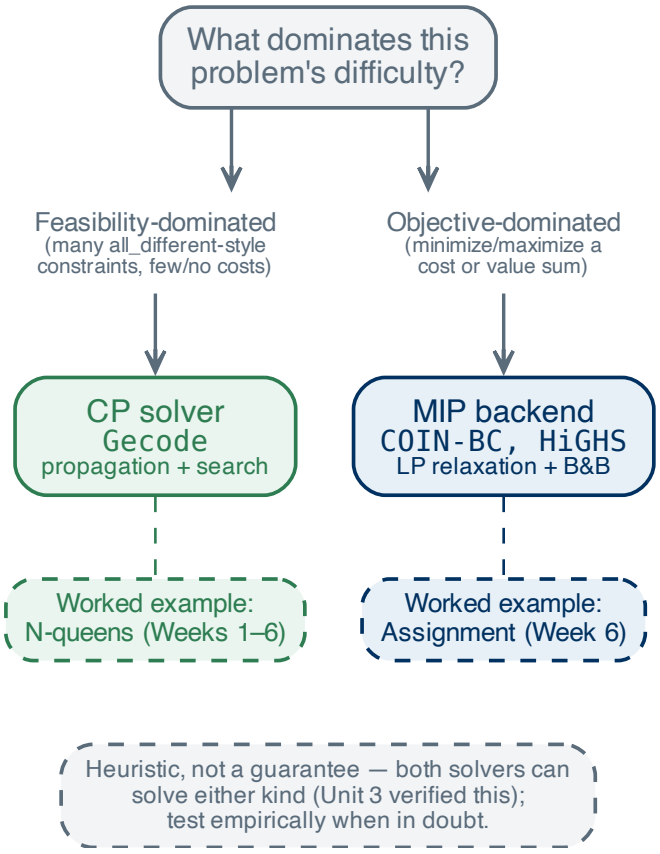


Figure 6.3: Choosing between a CP solver and a MIP backend: a structural decision guide anchored to this week's N-queens and assignment-problem examples.

6.4.4 Now you try

3 • Participation **Now You Try**

In three sentences or fewer: would you expect N-queens (Weeks 1–4's `all_different`-heavy satisfaction problem) or the assignment problem (Unit 3's linear-cost optimization problem) to be the more natural fit for a CP solver like Gecode, and which for a MIP backend? Justify your answer using the decision guide's two structural signals – constraint-heavy feasibility versus a linear objective.

Feedback. N-queens is the more natural CP fit: it is a pure satisfaction problem dominated by combinatorial `all_different`-style constraints with no linear objective at all, exactly Gecode's propagation-plus-search strength. The assignment problem is the more natural MIP fit: its feasibility constraint (`all_different` on the assignment array) is simple, but its defining feature is a linear objective over a cost matrix, exactly what LP relaxation and branch-and-bound are built for. Both solvers can, and in Unit 3 did, solve either kind correctly – this is a "which is the more natural fit" question about problem structure, not a "which one works" question.

Check Yourself 6.4

Using this week's structural decision guide, would you expect a large bin-packing instance (many combinatorial packing constraints, no natural linear cost) to be a more natural fit for Gecode or for a MIP backend – and why?

5 • Follow-through **Transfer — Where Else?**

The `Model/Instance/Solver.lookup` pattern you just used to script a comparison across Gecode and a MIP backend is the same Python-side habit Week 7 builds on when it introduces `gurobipy` as a second, complementary way of driving Gurobi directly, rather than through MiniZinc. The structural CP-versus-MIP decision guide you built here – does feasibility or the objective dominate this problem's difficulty – is a question worth asking about every scheduling, routing and rostering problem in later weeks, long after this week's specific knapsack and assignment instances are gone from memory.

Chapter Summary

Summary

This chapter separated, for the first time in this course, the model you build from the program that solves it. Unit 1 introduced MiniZinc as a declarative language, walked through its four building blocks – parameters, decision variables, constraints and the `solve` item – and used them to build a complete 0/1 knapsack model, including the model/data separation between a `.mzn` file and a `.dzn` file. Unit 2 opened the compilation step every model passes through before any solver sees it: flattening translates a `.mzn` model into FlatZinc, substituting pa-

rameter values, mapping global constraints such as `all_different` to a target solver's own predicate names, and introducing auxiliary variables you never wrote – confirmed against a real, captured excerpt of the Week 1–4 N-queens model's compiled `.fzn` file. Unit 3 ran the exact same `assignment.mzn` file, unedited, against Gecode, COIN-BC and HiGHS, all three reporting the identical optimal cost of 13 at different wall-clock speeds and by genuinely different algorithms – and against Gurobi, which is registered by `minizinc --solvers` in this environment but fails outright with a real license error, the concrete evidence behind this unit's central warning that "listed" and "usable" are two different claims. Unit 4 moved that same comparison into Python with the `minizinc` package's `Model`, `Instance` and `Solver.lookup` objects, and built a short, structural decision guide – feasibility-dominated problems tend to suit a CP solver, objective-dominated linear problems tend to suit a MIP backend – to be tested empirically on each new problem, not applied as a fixed rule. Next week keeps both of this week's problems – knapsack and assignment – but opens the MIP backend directly: you will write linear and integer programs by hand, understand LP relaxation and branch-and-bound as the mechanics that made COIN-BC and HiGHS work this week, and build your first `gurobipy` models, a second full modeling paradigm standing alongside CP-SAT and MiniZinc for the rest of the course.

Check Yourself Answers

Question (Unit 1 – parameter versus decision variable in the knapsack model). A parameter is a fixed, known quantity, typically supplied by a `.dzn` file – its value is data the model receives, never a value the solver chooses. A decision variable, declared with `var`, is an unknown the solver assigns a value to from within a stated domain. `capacity` is a parameter: its numeric value is given as data before solving and is never assigned by the solver, unlike the `array[1..n] of var bool: x` decision variables, which the solver determines.

Question (Unit 2 – why `all_different` does not survive flattening). Flattening translates every high-level MiniZinc construct, including global constraints, into whichever FlatZinc predicate the target solver's own library defines for it – `fzn_all_different_int`, confirmed for Gecode's target in this environment's own captured `.fzn` output. FlatZinc is a fixed, solver-agnostic intermediate format, not a copy of MiniZinc's own vocabulary, so the literal identifier a modeler wrote in the `.mzn` source is not expected to reappear unchanged in the compiled file.

Question (Unit 3 – Gecode versus HiGHS on the assignment problem). Guaranteed identical: the optimal objective value (total cost) and feasibility or infeasibility. Not guaranteed: solving time, which specific optimal assignment is reported if several tie for the best cost, and the internal search trace – Gecode's propagation-plus-search and HiGHS's LP-relaxation-plus-branch-and-bound are different algorithms that happen to agree on the answer, not the same algorithm run twice.

Question (Unit 4 – a large bin-packing instance: Gecode or a MIP backend?). Gecode. Bin packing of this kind is dominated by combinatorial feasibility constraints – which items fit together into which bins – rather than a linear objective,

matching the feasibility-dominated, combinatorial signal the decision guide associates with CP solvers. A MIP formulation of bin packing is also possible, but the structural signal points to CP as the more natural first choice, one still worth testing empirically the way Unit 3 tested Gecode against COIN-BC and HiGHS on the assignment problem.